

UNIVERSITÉ HASSAN II DE CASABLANCA
FACULTÉ DES SCIENCES BEN M'SICK
MASTER DATA SCIENCE ET BIG DATA

Analyse Comparative : Métaheuristiques vs Descente de Gradient

Application à la classification sur le dataset Iris

Réalisé par :
Achraf Chahid

Année Universitaire :
2025 - 2026

15 juin 2026

Table des matières

1	Introduction	2
2	Méthodologie et Architecture	2
2.1	Données et Modélisation Mathématique	2
2.2	Les Algorithmes Déployés	2
2.3	Architecture Logicielle Découplée (Full-Stack)	3
3	Résultats Expérimentaux	3
3.1	Synthèse des Métriques	3
3.2	Analyse Cinétique de la Convergence	3
4	Simulation Avancée et Limites Analytiques	4
4.1	Visualisation des Trajectoires sur la Fonction de Rastrigin	4
4.2	La Limite Inhérente : Le Passage à l'Échelle (Scalability)	5
5	Conclusion	5

1 Introduction

L'entraînement des réseaux de neurones profonds repose fondamentalement sur la minimisation d'une fonction de perte dans un espace multidimensionnel. Historiquement, les méthodes d'optimisation basées sur le gradient, telles que la descente de gradient stochastique (SGD) et ses variantes adaptatives (Adam), constituent le standard industriel. Elles s'appuient sur la dérivabilité de la fonction objectif pour converger rapidement vers un minimum local.

Cependant, face à des fonctions non lisses, non différentiables, ou des paysages d'optimisation fortement non convexes criblés de minima locaux trompeurs, ces méthodes peuvent échouer. C'est dans ce contexte que les métaheuristiques, inspirées de processus naturels, offrent une alternative robuste. L'optimisation par essaim particulaire (PSO - *Particle Swarm Optimization*) en est un excellent représentant, privilégiant une exploration globale stochastique plutôt qu'une exploitation purement locale.

L'objectif de ce projet est de réaliser une étude empirique comparant rigoureusement SGD, Adam et PSO. Pour éviter la malédiction de la dimensionnalité qui pénalise dramatiquement les méthodes populationnelles comme PSO, l'étude s'appuie sur un problème de complexité maîtrisée : la classification multiclasse du dataset Iris. Afin d'enrichir l'analyse, cette implémentation algorithmique s'accompagne du développement d'une architecture full-stack (API et Interface Web) permettant la simulation visuelle de ces trajectoires d'optimisation.

2 Méthodologie et Architecture

2.1 Données et Modélisation Mathématique

Le dataset Iris (150 échantillons, 4 caractéristiques continues, 3 classes cibles) a été partitionné selon un ratio strict de 80/20 pour séparer les ensembles d'apprentissage et de test. Une standardisation (Z-score) a été appliquée en amont pour assurer la stabilité numérique des gradients.

Afin d'isoler véritablement l'impact de l'algorithme d'optimisation sur l'apprentissage, un Perceptron Multicouche (MLP) minimaliste a été modélisé en utilisant le framework PyTorch. L'architecture se définit comme suit :

- Une couche d'entrée $X \in \mathbb{R}^4$.
- Une couche cachée dense composée de 10 neurones, activée par une fonction non linéaire ReLU : $h = \max(0, W_1X + b_1)$.
- Une couche de sortie $Y \in \mathbb{R}^3$, dont la perte est évaluée via l'entropie croisée (*Cross-Entropy Loss*).

Ce modèle totalise exactement 83 paramètres optimisables (poids et biais). C'est ce vecteur $\theta \in \mathbb{R}^{83}$ que nos optimiseurs devront calibrer.

2.2 Les Algorithmes Déployés

1. **Stochastic Gradient Descent (SGD)** : Implémentation classique avec un taux d'apprentissage fixe. Il sert de *baseline* théorique.
2. **Adam (Adaptive Moment Estimation)** : Optimiseur de pointe qui calcule des taux d'apprentissage adaptatifs pour chaque paramètre en estimant les moments du premier (moyenne) et second ordre (variance non centrée) des gradients.

3. **Particle Swarm Optimization (PSO)** : Une métaheuristique développée "from scratch" en PyTorch. La population a été fixée à 30 particules. Chaque particule i navigue dans l'espace $\mathbb{R}^{83}$ en mettant à jour sa vitesse et sa position selon son meilleur historique personnel et le meilleur historique global de l'essaim, sans jamais calculer la dérivée de la fonction de perte.

2.3 Architecture Logicielle Découplée (Full-Stack)

Au-delà du script d'entraînement classique, ce projet s'est distingué par l'intégration d'une architecture logicielle professionnelle, séparant la logique d'optimisation mathématique de sa restitution visuelle.

Le backend a été développé avec **FastAPI** pour exposer les trajectoires générées par les algorithmes sous forme de service web (API RESTful). Le frontend, construit avec **React** et **Tailwind CSS**, consomme ces données de manière asynchrone pour animer les trajectoires sur un tableau de bord analytique.

3 Résultats Expérimentaux

Les trois algorithmes ont été entraînés sur le dataset Iris pendant 100 itérations, dans des conditions matérielles strictement identiques (exécution CPU, initialisation des poids via *seed* fixe).

3.1 Synthèse des Métriques

Le tableau 1 expose les performances finales. La dichotomie entre la précision mathématique et le coût computationnel y est évidente.

Optimiseur	Temps d'exécution (s)	Perte Finale (Train)	Précision (Test)
SGD	0.179	0.94100	70.0%
Adam	0.155	0.08361	100.0%
PSO	0.425	0.03781	100.0%

TABLE 1 – Comparaison empirique de la qualité de la solution et du coût computationnel

3.2 Analyse Cinétique de la Convergence

Pour comprendre ces métriques, l'observation des courbes de convergence est primordiale.

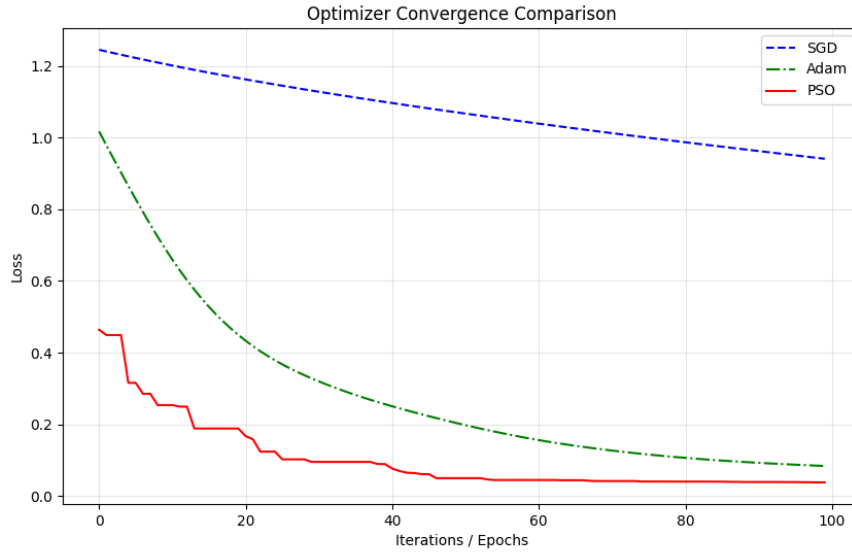


FIGURE 1 – Évolution de la fonction de perte (Cross-Entropy) au fil des itérations

L'analyse de la figure 1 révèle trois comportements structurellement différents :

- **La rigidité de SGD** : Avec un pas d'apprentissage fixe de 0.01, SGD peine à descendre efficacement le long des ravins de la fonction de perte. En 100 itérations, il n'atteint qu'une précision médiocre (70%).
- **La fulgurance d'Adam** : Grâce à sa capacité à accumuler de l'inertie (Momentum) et à moduler son apprentissage, Adam descend la surface d'erreur avec une fluidité remarquable, atteignant rapidement un excellent minimum local.
- **L'exploration en paliers de PSO** : La courbe rouge de PSO démontre la nature stochastique de l'algorithme. La fonction de perte stagne pendant que l'essaim explore, puis chute brutalement lorsqu'une particule découvre une meilleure région. In fine, PSO trouve un minimum global **plus bas que celui d'Adam** (0.037 vs 0.083).

4 Simulation Avancée et Limites Analytiques

4.1 Visualisation des Trajectoires sur la Fonction de Rastrigin

Pour pallier l'impossibilité de visualiser l'espace en 83 dimensions du modèle Iris, nous avons conçu un outil de simulation web. L'API projette les algorithmes sur la **fonction de Rastrigin en 2D**, une surface mathématique délibérément criblée de minima locaux pièges.

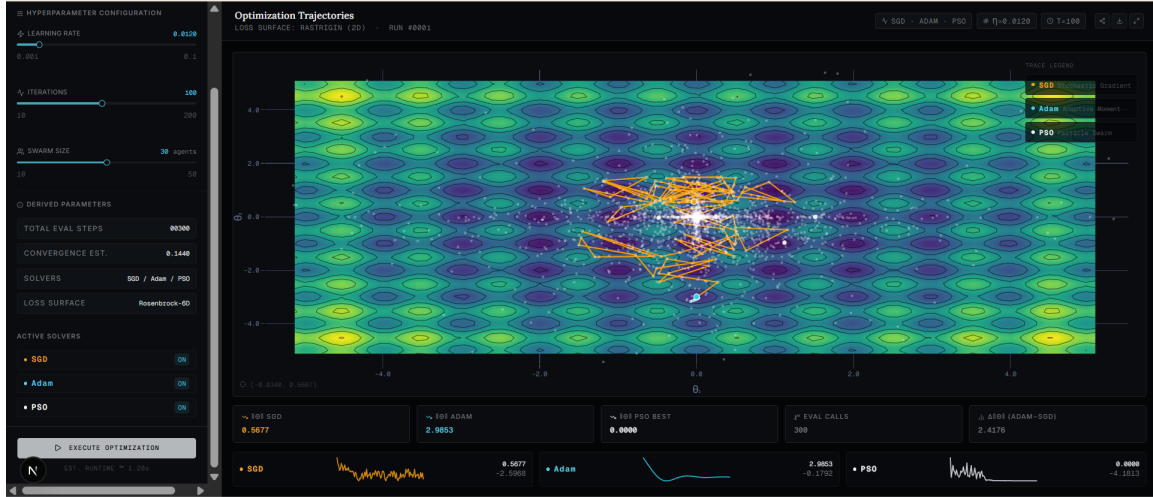


FIGURE 2 – Tableau de bord interactif (React/Plotly) illustrant l'évitement des minima locaux

Comme l'illustre l'interface analytique (Figure 2), les méthodes de gradient strict (SGD, Adam) ont une forte propension à converger vers le minimum local le plus proche de leur point d'initialisation. À l'inverse, l'essaim particulaire (PSO) disperse ses agents. Si certaines particules sont piégées, elles sont finalement "rappelées" par l'influence gravitationnelle de la particule ayant découvert le véritable minimum global.

4.2 La Limite Inhérente : Le Passage à l'Échelle (Scalability)

Si PSO triomphe sur la qualité de la solution finale dans un espace restreint ($\mathbb{R}^{83}$), le tableau 1 trahit sa principale faiblesse : le temps de calcul. PSO nécessite ici 0.425 secondes contre 0.155 secondes pour Adam.

Dans un algorithme de gradient, chaque lot de données (batch) ne requiert qu'une seule passe aller-retour (Forward/Backward propagation) pour ajuster le modèle complet. Pour PSO, sans calcul de gradient, le modèle doit évaluer la fonction de perte par *Forward pass* complète pour **chaque particule** à **chaque itération**. Sur un réseau de grande envergure (ex : ResNet, Transformers avec des millions de paramètres), l'espace de recherche devient trop vaste. L'exploration stochastique par essaim subit la "malédiction de la dimensionnalité", rendant la convergence non seulement atrocement lente, mais souvent impossible.

5 Conclusion

Ce projet d'ingénierie appliquée confirme qu'en matière d'optimisation mathématique pour le Machine Learning, la loi du "No Free Lunch" prévaut.

- Les **méthodes basées sur le gradient**, notamment les variantes adaptatives comme Adam, sont indispensables à l'ère du Deep Learning. Leur rapport coût-calcul/efficacité est imbattable sur des fonctions différentiables de très grande dimension.
- Les **métaheuristiques populationnelles**, comme PSO, offrent une résilience fascinante face aux surfaces non convexes et non dérivables. Bien qu'inadaptées à l'entraînement direct des poids de grands réseaux de neurones, elles restent des outils

redoutables pour des tâches d'exploration complexe, comme l'optimisation des hyperparamètres (Hyperparameter Tuning) ou la conception d'architecture neuronale (Neural Architecture Search).

Enfin, le déploiement de ces mécanismes d'optimisation au sein d'une architecture applicative distribuée (FastAPI / React) a démontré l'importance cruciale de coupler la rigueur des Data Sciences aux bonnes pratiques du Software Engineering (Data Engineering) pour rendre les modèles explicables et accessibles.